Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования

«Алтайский государственный технический университет им. И. И. Ползунова»

Факультет информационных технологий Кафедра информатики, вычислительной техники и информационной безопасности

Отчет защищен с оценкой_____
Преподаватель _____С. В. Умбетов
«____»_____2026 г.

Отчёт по лабораторной работе №3 по дисциплине «Основы компьютерных сетей» «Анализ сетевых портов и активных соединений»

ЛР 09.02.08.23.003

Студент группы 1ИИС-41 М.А.Синтиков
группа и.о., фамилия
Преподаватель старший преподаватель, доцент С. В. Умбетов
должность, ученая степень и.о., фамилия

Барнаул 2026

Лабораторная работа №3
Анализ сетевых портов и активных соединений

Цель работы: Изучение механизмов работы сетевых портов и сокетов, освоение утилит анализа активных соединений и слушающих портов: ss, netstat и lsof. Студенты научатся запускать простейшие сетевые службы, определять, какие порты открыты, какие процессы их занимают, и выявлять конфликты портов. Работа выполняется на Ubuntu Server в VirtualBox (лабораторная работа №1) и на локальной машине.

Задачи к работе:

1)С помощью утилиты ss вывести список всех слушающих портов на Ubuntu Server и определить, какие службы соответствуют обнаруженным портам (22, 53 и другим).

2)Установить пакет net-tools и сравнить вывод команды netstat -tulnp с выводом ss. Подробно описать различия между утилитами.

3)Запустить простой HTTP-сервер на Python (python3 -m http.server 8000) и с помощью ss и lsof подтвердить, что порт 8000 находится в состоянии LISTEN и принадлежит процессу python.

4)Смоделировать конфликт портов, запустив второй HTTP-сервер на том же порту 8000. Зафиксировать сообщение об ошибке, найти PID первого процесса, завершить его и успешно запустить сервер снова.

5)Запустить netcat в режиме прослушивания на порту 9999, проверить появление порта через ss/lsof, подключиться к нему с другого терминала и зафиксировать изменение состояния сокета.

6)Вывести все ESTABLISHED-соединения, найти среди них SSHподключение и записать локальный и удалённый адрес/порт.

7) С помощью ss или netstat посчитать количество сокетов в состоянии TIME_WAIT и подробно объяснить механизм их возникновения.

Задание принято:



Ситников М.А. Подпись

Ход работы

1. С помощью ss -tulnp выведите список всех слушающих портов на Ubuntu Server. Определите, какие службы соответствуют портам 22, 53 (если есть) и другим обнаруженным портам.

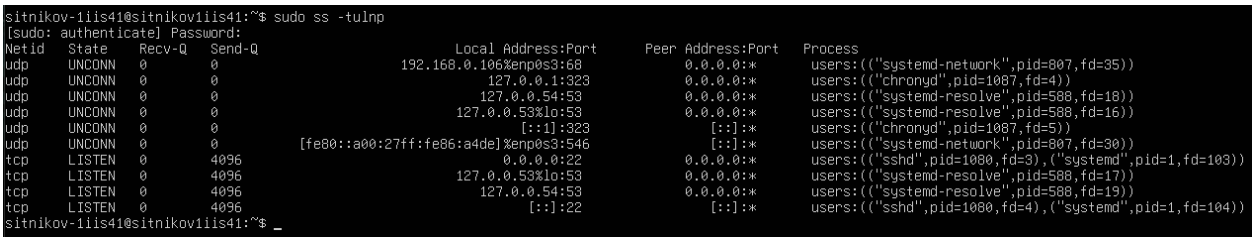


Рис.1 – Результат выполнения команды sudo ss –tulnp на Ubuntu server

Результат:
Порт 22 (TCP) : процесс sshd (PID 1080). Это SSH-сервер, обеспечивающий удалённое подключение к серверу. Порт входит в диапазон привилегированных (0–1023) и требует прав root для прослушивания.
Порт 53 (UDP/TCP) : процесс systemd-resolved (PID 588). Это локальный DNS-резолвер, обслуживающий запросы на разрешение доменных имён.
Порт 68 (UDP) : процесс systemd-networkd (PID 807). Это DHCP-клиент, который получает IP-адрес и другие сетевые настройки (шлюз, DNS) от DHCP-сервера. Для прослушивания требуются права root.

2. Установите net-tools (если нужно) и сравните вывод netstat -tulnp с выводом ss. Кратко опишите различия.

```
Get:7 http://ru.archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [110 kB]
Get:8 http://ru.archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [542 kB]
Get:9 http://ru.archive.ubuntu.com/ubuntu resolute-updates/restricted Translation-en [112 kB]
Get:10 http://ru.archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [334 kB]
Get:11 http://ru.archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [111 kB]
Get:12 http://ru.archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [201 kB]
Get:13 http://ru.archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Components [1056 B]
Get:14 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [607 kB]
Get:15 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [141 kB]
Get:16 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.8 kB]
Get:17 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [206 kB]
Get:18 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [67.9 kB]
Get:19 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [55.5 kB]
Fetched 3921 kB in 1s (3430 kB/s)
49 packages can be upgraded. Run 'apt list --upgradable' to see them.
sitnikov-1iis41@sitnikov1iis41:~$ sudo apt install net-tools -y
Installing:
  net-tools

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 49
  Download size: 151 kB
  Space needed: 680 kB / 4729 MB available

Get:1 http://ru.archive.ubuntu.com/ubuntu resolute/main amd64 net-tools amd64 2.10-2ubuntu1 [151 kB]
Fetched 151 kB in 0s (554 kB/s)
Selecting previously unselected package net-tools.
(Reading database ... 100995 files and directories currently installed.)
Preparing to unpack .../net-tools_2.10-2ubuntu1_amd64.deb ...
Unpacking net-tools (2.10-2ubuntu1) ...
Setting up net-tools (2.10-2ubuntu1) ...
Processing triggers for man-db (2.13.1-1build1) ...
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

Restarting services...

Service restarts being deferred:
 systemctl restart networkd-dispatcher.service
 systemctl restart unattended-upgrades.service

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
sitnikov-1iis41@sitnikov1iis41:~$
```

Рис.2 – Установка Net-tools на Ubuntu server

```
sitnikov-1iis41@sitnikov1iis41:~$ sudo netstat -tulnp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      1/systemd
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      588/systemd-resolve
tcp        0      0 127.0.0.54:53           0.0.0.0:*               LISTEN      588/systemd-resolve
tcp6       0      0 :::22                   :::*                    LISTEN      1/systemd
udp        0      0 192.168.0.106:68        0.0.0.0:*                           807/systemd-network
udp        0      0 127.0.0.1:323           0.0.0.0:*                           1087/chronyd
udp        0      0 127.0.0.54:53           0.0.0.0:*                           588/systemd-resolve
udp        0      0 127.0.0.53:53           0.0.0.0:*                           588/systemd-resolve
udp6       0      0 ::1:323                 :::*                                1087/chronyd
udp6       0      0 fe80::a00:27ff:fe86:546 :::*                                807/systemd-network
sitnikov-1iis41@sitnikov1iis41:~$
```

Рис.3 – Выполнение команды sudo netstat –tulnp на Ubuntu server

Результат:
Источник данных: Утилита ss получает информацию напрямую из ядра Linux через netlink-сокеты. Утилита netstat читает текстовые файлы в виртуальной файловой системе /proc/net/.
Производительность: На системах с большим числом соединений ss работает значительно быстрее, так как не выполняет парсинг текстовых файлов. netstat в таких условиях может работать медленнее.
Функциональность и фильтры: ss поддерживает гибкие фильтры (например, sport = :80, dport = :443), тогда как netstat требует дополнительной обработки через grep.
Актуальность: netstat считается устаревшей утилитой и не развивается. В современных дистрибутивах Linux рекомендуется использовать ss.

3. Запустите python3 -m http.server 8000. С помощью ss и lsof подтвердите, что порт 8000 находится в состоянии LISTEN и принадлежит процессу python.

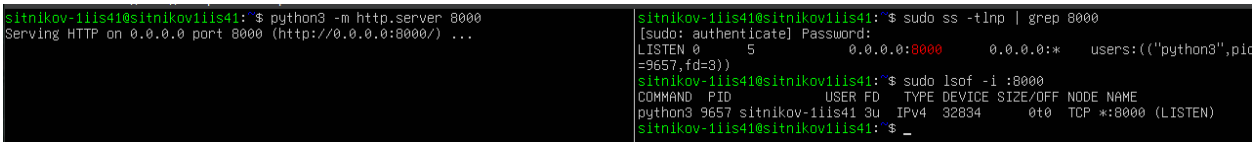


Рис.4 – Запуск python3 -m http.server 8000 в первом терминале и проверка состояния порта 8000 через ss и lsof во втором терминале Ubuntu server

Результат:
sudo ss -tlnp | grep 8000 — утилита показала, что сокет на порту 8000 находится в состоянии LISTEN, а в качестве процесса-владельца указан python3 с идентификатором PID=9657.
sudo lsof -i :8000 — команда подтвердила, что процесс с именем python3 (PID 9657) открыл сетевой сокет (тип IPv4) на порту 8000 и удерживает его в режиме прослушивания (LISTEN).
Обе команды выдали одинаковую информацию: порт 8000 успешно занят сервером python3, сокет активен и готов принимать входящие HTTP-запросы.

4. Попробуйте запустить второй HTTP-сервер на том же порту 8000. Зафиксируйте сообщение об ошибке. Найдите PID первого процесса и завершите его, после чего успешно запустите сервер снова.

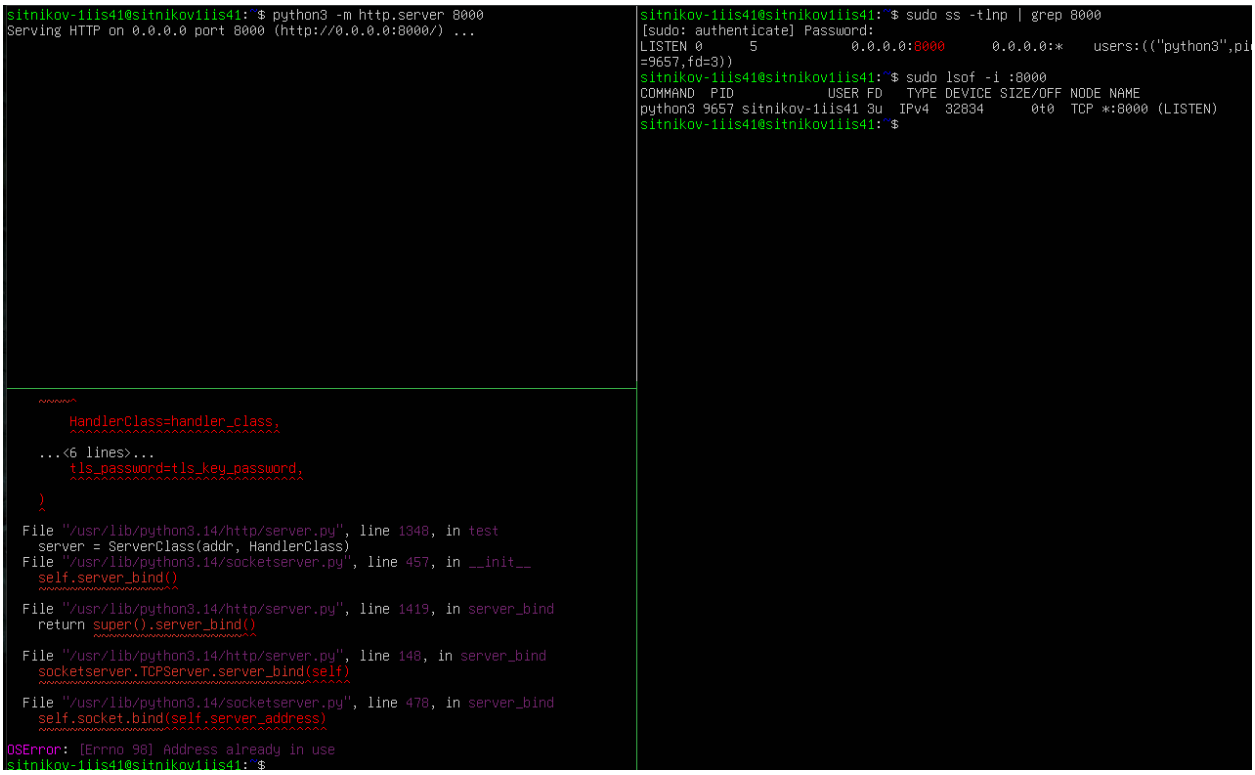


Рис.5 – Попытка запустить второй HTTP-сервер на том же порту завершилась ошибкой Address already in use

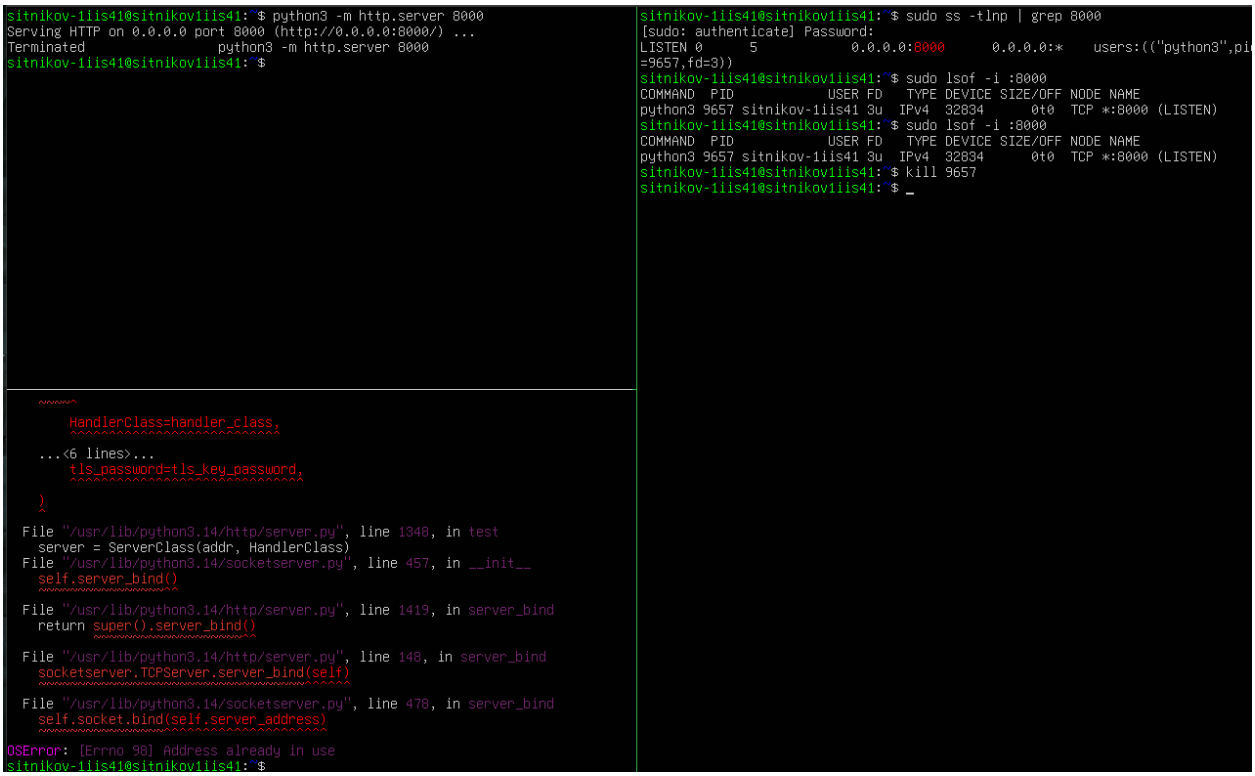


Рис.6 - Поиск PID через lsof и завершение процесса.

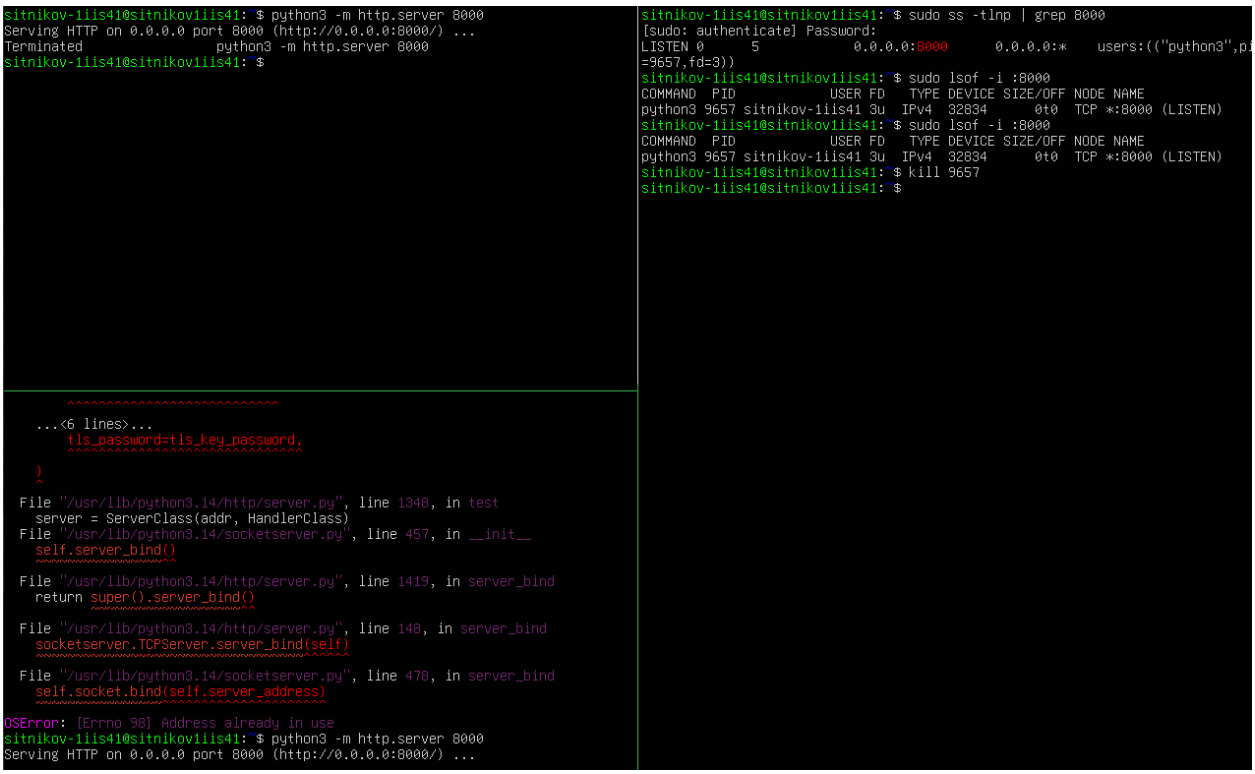


Рис.7 – Успешный перезапуск сервера

Обоснование ошибки: Операционная система не позволяет двум процессам одновременно слушать один и тот же TCP-порт на одном сетевом интерфейсе. При поступлении входящего пакета на порт 8000 сетевой стек ядра не смог бы однозначно определить, какому именно процессу его доставить. Подобное поведение является защитным механизмом, предотвращающим конфликты между сетевыми службами и нарушение логики работы стека.

Решение проблемы:

1. С помощью команды sudo lsof -i :8000 я определил PID первого процесса python3, который удерживал порт. В моем случае это был PID 9657.
2. Завершение конфликтующего процесса. Выполнил команду kill 9657. После этого в первом терминале работа HTTP-сервера остановилась, и порт 8000 был освобожден.
3. Повторный запуск службы. Снова выполнил команду python3 -m http.server 8000. На этот раз сервер запустился успешно, так как конфликт портов был устранен.

5. Запустите netcat в режиме прослушивания на порту 9999. Проверьте появление порта через ss/lsof. Подключитесь к нему с другой машины или через второй терминал (nc 9999) и посмотрите, как изменится состояние сокета.

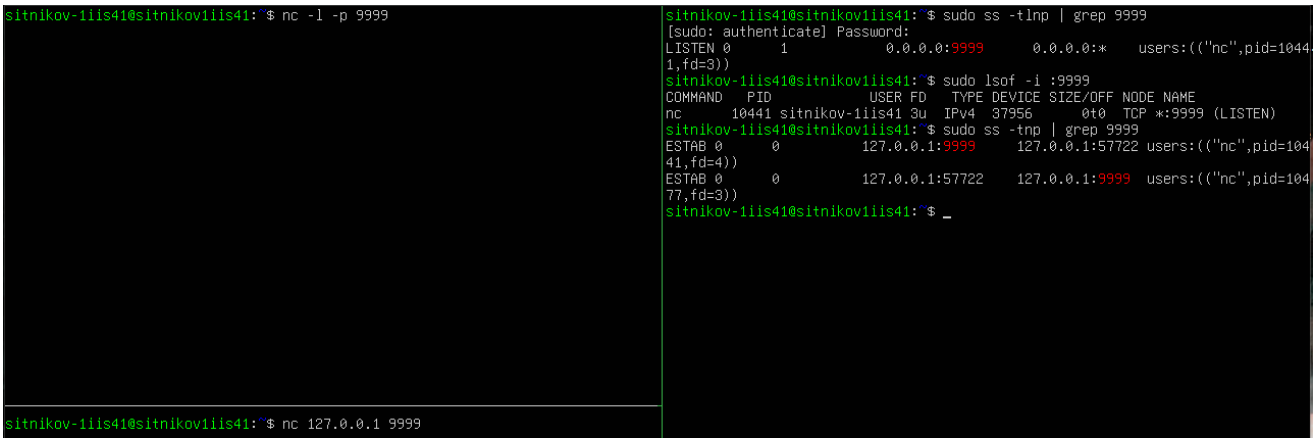


Рис.8 - Запуск nc -l -p 9999, подключение клиента nc, проверка состояния сокета командой sudo ss - tnp | grep 9999 (без флага -l, чтобы видеть все соединения)

Обоснование смены состояния: До момента подключения клиента сокет на серверной стороне (порт 9999) находился в состоянии LISTEN. После подключения сокет перешёл в состояние ESTABLISHED, что означает активное установленное соединение, готовое к обмену данными.

6. Выведите все ESTABLISHED-соединения. Найдите среди них своё SSH-подключение и запишите локальный и удалённый адрес/порт.

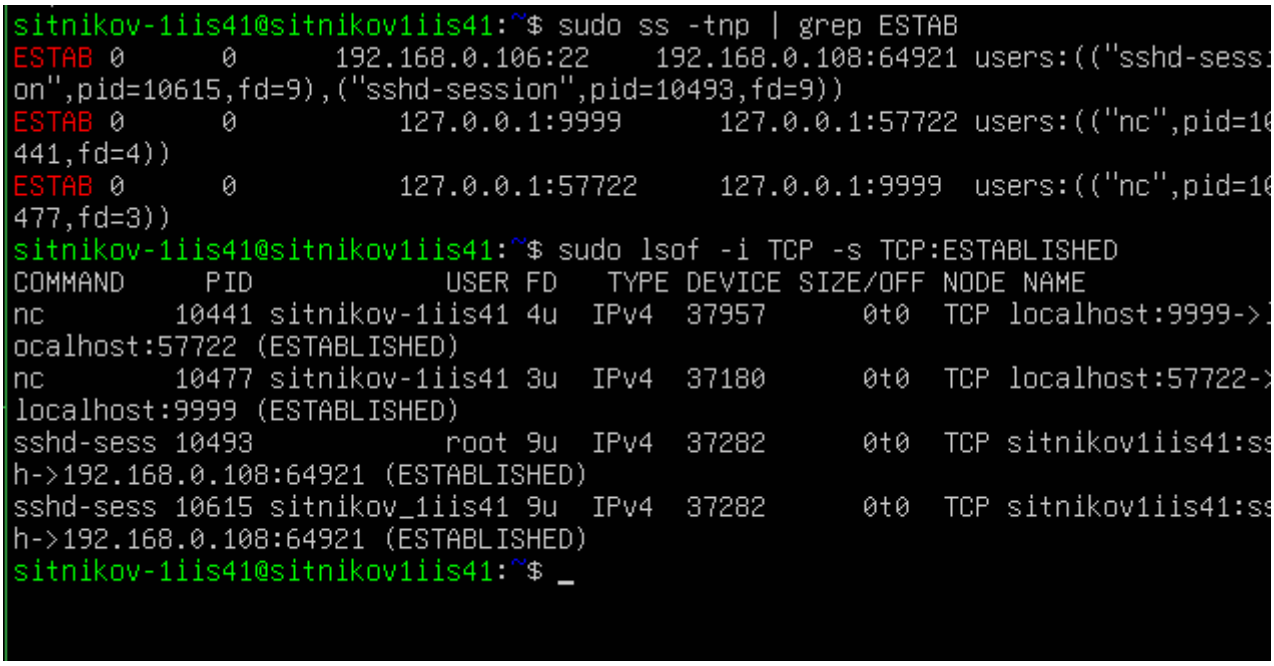


Рис.9 – Вывод команды ss и lsof с активными соединениями

Результат:
SSH-подключение (мой текущий сеанс из PuTTY):
Локальный адрес: 192.168.0.106:22 (порт SSH на сервере, привилегированный порт).
Удалённый адрес: 192.168.0.108:64921 (шлюз/адрес локальной машины, с которой выполнено подключение, с эфемерным портом 64921).
Процесс: sshd-session (PID 10493 и 10615, от пользователя root и sitnikov_liis41).
Состояние: ESTABLISHED — активное соединение, идёт обмен данными.
Netcat-соединение:
Серверная сторона: localhost:9999 -> localhost:57722, PID процесса nc (10441), дескриптор 4u.
Клиентская сторона: localhost:57722 -> localhost:9999, PID процесса nc (10477), дескриптор 3u.
Состояние: ESTABLISHED (соединение установлено).
Анализ:
SSH-соединение использует привилегированный порт 22 на стороне сервера (192.168.0.106) и эфемерный порт 64921 на стороне клиента (192.168.0.108, локальная машина). Netcat-соединение полностью локальное (127.0.0.1): сервер слушает порт 9999, клиент использует эфемерный порт 57722. Оба

соединения находятся в состоянии ESTABLISHED, что подтверждает успешный обмен данными.

7. (Дополнительно) С помощью ss или netstat посчитайте, сколько сокетов находится в состоянии TIME_WAIT. Объясните, откуда они берутся.

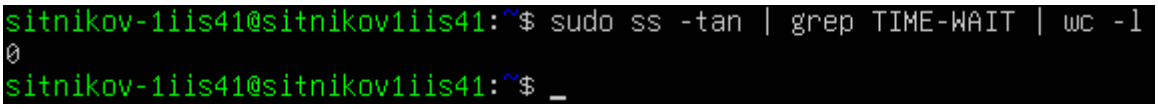


Рис.10 – Подсчёт сокетов в состоянии TIME_WAIT командой ss

Результат: 0. Это значит, что на момент проверки таких сокетов на сервере не было — все соединения уже полностью закрылись, и время ожидания истекло.

Что такое TIME_WAIT:

Когда одна из сторон решает закрыть TCP-соединение (отправляет первый FIN), после завершения обмена пакетами (FIN и ACK) сокет не исчезает. Он зависает в состоянии TIME_WAIT примерно на 60 секунд (это время 2 × MSL — максимальное время жизни пакета в сети).

Зачем TIME_WAIT нужен:

1. Чтобы не потерять последнее подтверждение. Если последний ACK от сервера не дойдёт до клиента, клиент повторит FIN. Сокет в TIME_WAIT сможет спокойно принять его и ответить ещё раз, завершив соединение корректно.
2. Чтобы старые пакеты не попали в новое соединение. В сети могут «блуждать» запоздавшие пакеты от уже закрытого соединения. Если сразу открыть новое соединение с теми же портами, эти старые пакеты могут быть приняты за новые. TIME_WAIT даёт сети время очиститься.
3. На серверах, которые обрабатывают много коротких запросов (например, веб-серверы), такие сокеты могут копиться. Это приводит к нехватке свободных портов. Чтобы этого избежать, используют настройку tcp_tw_reuse (разрешает переиспользовать порты) или keep-alive соединения (чтобы не открывать и не закрывать связь постоянно).

Ссылка на GitHub:

https://github.com/Enot2130/Sitnikov_1iis41/tree/main/Основы%20компьютерных%20сетей/Анализ%20сетевых%20портов%20и%20активных%20соединений

Контрольные вопросы

1. Что такое порт и зачем он нужен?
Это число (0–65535), которое помогает компьютеру понять, какому именно приложению (сайту, SSH и т.д.) предназначен входящий интернет-пакет. Вместе с IP-адресом порт образует сокет. Порт нужен для того, чтобы на одном IP-адресе могло одновременно работать несколько сетевых служб.

2. Какие бывают диапазоны портов?
- 0–1023 — системные (SSH 22, HTTP 80). Для запуска нужны права администратора (root).
- 1024–49151 — порты для обычных программ.
- 49152–65535 — временные (динамические), выдаются клиентам на время соединения.

3. Что значит состояние LISTEN?
Программа запущена, «слушает» порт и ждет, когда к ней кто-нибудь подключится.

4. Чем ESTABLISHED отличается от TIME_WAIT?
ESTABLISHED — соединение активно, идет обмен данными.
TIME_WAIT — соединение уже закрыто, но программа еще минуту держит его в памяти, чтобы обработать запоздавшие пакеты.

5. Почему ss лучше netstat?
ss быстрее, потому что берет данные напрямую из ядра. netstat — устаревший и медленный, он читает текстовые файлы.

6. Какие ключи ss показывают порты и процессы?
Команда sudo ss -tulnp:
-t и -u — TCP и UDP.
-l — только слушающие.
-n — показывать номера, а не имена.
-p — показать процесс (PID), которому принадлежит порт.

7. Как через lsof узнать, кто занял порт 8080?
Ввести команду sudo lsof -i :8080. Она покажет имя программы и её PID.

8. Почему два процесса не могут слушать один TCP-порт?

Компьютер не сможет понять, какому из двух процессов отдавать входящий пакет. Это защита от конфликтов.

9. Как решить конфликт портов?

1. Найти процесс: sudo lsof -i :8000 или ss -tlnp | grep 8000;
2. Завершить его: kill (или kill -9 при необходимости);
3. Либо запустить новую службу на другом порту.

10. Что такое эфемерные порты в списке соединений?

Это временные порты (от 49152), которые система выдает клиенту для соединения. Они используются на время сеанса связи и освобождаются после его закрытия.

11. Зачем нужны права root?

Без них утилиты (ss, lsof) покажут только ваши процессы. Системные службы (sshd, systemdresolved, nginx и т.п.) будут скрыты или без имен. С sudo видно всё.

12. Как отличить клиентский сокет от серверного в выводе ss?

Серверный — в состоянии LISTEN, из диапазона well-known (0–1023) или registered (1024– 49151), например, :22, :80, :8000.
Клиентский — в состоянии ESTABLISHED, TIME_WAIT и др., но не LISTEN, локальный порт временный (49152–65535), а удаленный — известный (например, 22 или 80).

Вывод

В ходе лабораторной работы я освоил утилиты ss, netstat и lsof для анализа сетевых портов и соединений в Linux. Я научился находить слушающие порты и определять их службы (SSH на 22, DNS на 53, DHCP на 68), а также сравнивать ss и netstat — убедился, что ss быстрее и информативнее. На практике я запускал простые службы (Python HTTP-сервер, netcat) и проверял их состояние через LISTEN и ESTABLISHED, выявлял и решал конфликты портов (находил PID через lsof и завершал процесс командой kill), анализировал активные соединения, включая собственный SSH-сеанс, и разобрался с механизмом TIME_WAIT и его ролью в надёжном закрытии TCP-соединений.